

本体的用处是规定必须填哪些字段 Ontology-supported AI Model and Dataset Management

本篇范围说明：这是一篇 6 页的会议论文（FZI 信息技术研究中心与蒂宾根大学，Jan Novacek 等六人）。

要注意它的时间——原文是 IEEE INDIN 2024 的正式发表版（DOI 10.1109/INDIN58382.2024.10774524），2026-08-21 才挂到 arXiv（2608.21224v1）。所以它描述的技术生态是 2024 年的状态，读的时候要按两年前的语境理解。下面「全文翻译」覆盖摘要、§1 引言、§2 相关工作、§3 交换平台、§4 本体、§5 评估用例、§6 结论；图 4 那段完整的 Turtle 元数据示例节选关键行说明结构，不逐行译；参考文献不译。

核心内容

Q：这篇论文要解决什么问题？ A：AI 模型在企业之间传递时，接收方看不懂它。

作者用汽车行业举例：AI 模型和相关资产在供应链的各级供应商（tier）之间转手，而**主要障碍是缺少带显式语义的结构化元数据，以及对「这种元数据格式该长什么样」的共同理解**。

Hugging Face 这类社区托管了大量模型和数据集，但资源信息是**半结构化描述**——主要是自然语言文本，内容和篇幅都不统一；有些模型有结构化元数据，多数没有。而 MLflow、H2O、Ray 这类机器学习平台虽然都能给模型加元数据，**但元数据规范之间缺乏互操作性**，跨公司交换时格式和内容各不相同，问题就更明显。

研究问题被写成一句话：**「在工业语境下协作开发和使用一个 AI 模型，需要什么？」**

Q：他们做了什么？ A：两样东西，可以分开用：

- **AIMDEP** (AI Model and Dataset Exchange Platform)：一个协作开发与使用 AI 模型的平台，提供中央注册表，让模型和数据集对所有用户可见可访问。
- **AIMDEO** (AI Model and Dataset Exchange Ontology)：一个用 OWL 表达的本体，专门描述机器学习资产的元数据。

平台的技术栈很具体：客户端-服务器架构，服务端和 Web UI 用 Django 框架，另有 REST API 供自定义客户端调用；搜索用 OpenSearch、数据集可视化用 Plotly、模型部署用 MLEM、推理界面用 Gradio 自动生成（根据模型的 task/subtask 和输入输出特征）。

有一个工程设计点被埋在正文里，值得单独拎出来：**借 Django 的 MVT (Model-View-Template) 架构，本体的更新可以无缝地应用到服务端的 model 组件上，而不打断整体功能**。作者说这个设计确保了平台对未来本体更新的灵活性。

Q：资产是怎么注册进去的？ A：流程是**自动检测 + 人工核对**的混合，这个交互模式值得注意：

1. 上传物理文件后，平台**尝试自动识别访问和部署所需的后端**——数据集要识别数据处理器、模型要识别评估和部署所需的框架。支持的数据格式：CSV、Excel、JSON、Parquet；支持的框架：Scikit-Learn、TensorFlow、PyTorch。用户可以改平台选的框架。
2. **自定义的数据集和模型也能注册，但平台无法可视化或部署它们**——因为和资产交互必须依赖框架。
3. 输入输出特征**半自动定义**：平台尝试检测，但**用户应当核对并为每个特征补充元数据**。每个特征按 AIMDEO 存为一个 `Parameter`。作者说自动检测的一个好处是「降低出错的可能，例如避免遗漏特征」。
4. 用户在这一步加模型的配置参数，最后按 AIMDEO 补其余元数据（比如给模型加评估指标），然后注册进中央数据库。

Q: AIMDEO 这个本体规模多大、包含哪些概念？ A: 本体指标（论文的 Tab. 1）:

指标	数量
公理总数	414
逻辑公理数	178
声明公理数	105
类 (Class)	47
对象属性 (Object property)	29
数据属性 (Data property)	17
个体 (Individual)	5
标注属性 (Annotation property)	18
描述逻辑表达力	SHOIQ(D)

概念清单 (Tab. 2) 分三组，一共 14 个:

- **通用元数据**: Author (provenance 溯源信息)、Description (自然语言描述)、Name、Framework (执行模型所需的后端)
- **资产元数据**: Dataset (训练与测试用的数据)、Data source (数据集位置)、Training-/test-split (数据集划分)、Data points (数据集规模)
- **AI 模型刻画**: Task / sub-task (模型的预期用途)、Input、Output、Parameter、Quality criterion (质量标准)、Score (模型在某个指标上的得分)

Q: 四个协作场景是什么？ A: 这是本体设计的需求来源，作者先梳理协作场景再定概念。四个场景（论文的 Fig. 3）:

1. **内部使用 (Internal Use)**: 模型和数据集都留在创建者手里。模型在 AI 开发环境内被使用，结果存进产品开发环境，反馈再回到 AI 开发环境。
2. **共享数据 (Shared Data)**: 数据无法内部采集时，在供应链参与方之间交换数据。开发数据库里的数据被用于 AI 开发环境。**这种协作形式要求对数据访问有描述和良定义的接口。**

3. **共享模型 (Shared Models)**: 共享训练好的模型。与上一种相反, 产品开发环境的数据不共享, 共享的是模型。
4. **模型即服务 (Model Services)**: 不直接交换模型, 用户把请求发给一个服务, 服务内部委托给 AI 模型。

Q: 和已有工作的区别在哪? A: 论文对比了三个东西, 区别说得很清楚:

- **ITO** (Intelligence Task Ontology and Knowledge Graph, Blagec 等 2022 发表在 *Scientific Data*) —— 一个描述 AI 任务的本体和知识图谱。两者都支持规定模型的预期任务与子任务, 但 **AIMDEO** 额外包含描述 **provenance** (来源)、AI 模型种类、模型与数据集参数的概念, 还允许指定模型评估指标。而且 **AIMDEO** 直接复用 **ITO** 的标识符——图 4 里 `ns:hasTask` `"https://identifiers.org/ito:ITO_42033"` 就是引 **ITO** 的 IRI。
- **EMMM** (Experiment Management Meta-Model, Idowu 等 2022) —— 机器学习实验管理的元模型, 建了 `models`、`parameters`、`dependencies` 等核心概念, 还考虑了机器学习资产和传统资产的版本控制结构。但元素的元数据是用任意 **key-value** 映射捕获的, 缺乏共同格式和显式语义。这是本文的核心差异点。
- **工具调研** (Idowu 等 2022 的综述): 所有选项里只有 **MLflow** 支持资产注册和交换而不需要把数据共享给第三方, 但它不支持用本体描述元数据。**MLEM** 提供部署的标准接口和模型注册表, 但注册表很有限、不收集元数据。

Q: 「micro-ontology」是什么? A: 下载资产时, 除了原始文件, 平台还会把该资产的元数据按 **AIMDEO** 导出成 **micro-ontology** (微本体), 内容是属性及其取值。论文的图 4 给了完整例子, 是 Turtle 语法:

```
<.../aimodel/metric/metric0> a ns:AIModel ;
  ns:hasName "RF Instruction Cache-Line Access Classifier" ;
  ns:hasDescription "AI Model to classify single cache-line accesses
    to the instruction cache as hit or miss." ;
  ns:hasAIModelParameter <.../parameter/parameter0>, <.../parameter/parameter1> ;
  ns:hasAIModelInput <.../input/input0> ;
  ns:hasAIModelOutput <.../output/output0> ;
  ns:hasAIModelDataset <.../dataset/dataset0> ;
  ns:hasAIModelMetric <.../metric/metric0> ;
  ns:hasFramework "scikit-learn" ;
  ns:hasTask "https://identifiers.org/ito:ITO_42033" ;
  ns:hasVersion "1.0" .

<.../metric/metric0> a ns:AIModelMetric ;
  ns:hasName "average precision" ; ns:hasScore "0.9794" ; ns:hasUnit "dimensionless" .

<.../parameter/parameter0> a ns:AIModelParameter ;
  ns:hasName "Replacement-Strategy" ; ns:hasUnit "string" ; ns:hasScore "LRU" .
```

Q: 用例证明了什么？ A: 用例是预测内存访问时间——这在安全关键应用的软件时序预测里是必需的。三个角色接力：

- **领域专家**（这里是硬件开发者）建数据集，并用 **AIMDEO 把自己的隐性知识描述出来**——具体是缓存（内存层次）特性，如替换策略和大小。数据集注册到平台上。
- **AI 专家**能访问数据集**以及领域专家写下的那些知识**，这支持他设计合适的模型。模型做好后注册上去，AI 专家补充模型信息。
- **终端用户**（软件开发者）用平台的关键词搜索找合适的模型，用它分析自己的软件与内存配置，**从而评估是否需要采购测试硬件**。

例子里的模型是 **RF Instruction Cache-Line Access Classifier**，average precision 0.9794，参数 Replacement-Strategy=LRU、Cache-Size=2048 Byte，输入是 **set**（uint8，缓存行的组号）、输出是 **miss**（bool，命中还是未命中）。

作者强调的两处便利：**终端用户不必下载数据集就能用平台的分析功能**做快速分析、比较和选型；**模型能直接在平台上执行**，不用自己搭运行时环境，这降低了试用一个 AI 模型的门槛。

背景补充 / 点评

几个术语的位置

provenance（溯源） 在语义网领域是个有专门标准的概念（W3C PROV-O），指「一个资源是怎么来的、经过谁的手、用了什么」。这篇只把它落在 **Author** 一个概念上，是很轻的处理。

SHOIQ(D) 是描述逻辑（Description Logic）的表达力标记，每个字母代表一族构造子：**S** = 带传递角色的 ALC、**H** = 角色层次（`rdfs:subPropertyOf`）、**O** = 名义（nominals，即在类表达式里用个体）、**I** = 逆角色（inverse roles）、**Q** = 受限基数约束（qualified cardinality restrictions），**(D)** = 数据类型。这是相当强的表达力——**它的推理复杂度是 NEXPTIME**（非确定性指数时间）。工具（比如 Protégé）会自动算出本体用到的最高表达力并报出来。

MVT（Model-View-Template） 是 Django 对 MVC 模式的变体命名：Model 管数据结构与数据库映射、View 管业务逻辑、Template 管展示。这篇利用的正是「Model 层与数据库结构解耦」这一点。

MLEM 是 Iterative 公司（DVC 的团队）出的模型部署工具，提供跨框架的统一部署接口。这篇拿它做在线部署，也拿它的模型注册表做了对比（并指出它不收元数据）。

这篇的价值在哪、问题在哪

最值得记的是结论里那句话，它是本体最朴素也最站得住的价值主张：

平台简化了协作式 AI 模型开发，而**本体能帮助确保所需信息是可得的、以及这些信息该如何提供**。

注意这句话里**没有提推理、没有提知识发现、没有提智能问答**。它说的就是：把「必须填哪些字段、每个字段是什么意思」固定下来。这是一个很低调但很难被反驳的主张——**只要有两方要交换同一类东西，就需要一份双方都认的字段清单**。

这也解释了为什么它要拿 EMMM 做对比：EMMM 建了核心概念，但元数据是「任意 key-value 映射」。key-value 能装下任何东西，代价是**接收方不知道该期待哪些 key、也不知道某个 key 的值该怎么解释**。本体在这里做的事就是把 key 的集合和每个 key 的语义固定住。

第二个可学的是「复用现有标识符」这个做法。 `ns:hasTask`

`"https://identifiers.org/ito:ITO_42033"` ——不自己造任务分类，直接引一个已发布本体的 IRI。这是本体工程标准流程的第三步（确定范围 → 写能力问题 → **考虑复用现有词汇** → 定义类关系属性），这篇实打实做了。

四处问题

一、没有任何评估数据。「Evaluation」一节实际上是一次**叙述性的用例走查**——领域专家做什么、AI 专家做什么、终端用户做什么，全程没有量化指标：没有用户研究、没有「用了平台之后找到合适模型的时间从 X 降到 Y」、没有和 MLflow 或 Hugging Face 的对照实验。

作者的贡献第三条写的是 "Use case showing the **applicability** of the approach"——**applicability（可用性）而不是 effectiveness（有效性）**，这个措辞是诚实的。但它意味着这篇论文证明的是「这套东西能跑通一个场景」，**不是「它比现有做法好」**。引用这篇时不能把它当成「本体化元数据有效」的证据。

二、Individual count = 5 这个数字是个警示。47 个类、29 个对象属性，**但只有 5 个个体**。也就是说这个本体基本只有 TBox（概念层），几乎没有 ABox（实例层）——**它没有被真正填过数据**。一个声称要解决工业级资产交换的本体，实例数是 5，这和它的问题陈述规模严重不匹配。

三、SHOIQ(D) 这个表达力可疑。一个 47 个类的本体报出 SHOIQ 全套（含名义、逆角色、受限基数），意味着某处用到了这些构造子，但论文**没有说哪些公理用了哪些**。表达力高不等于用得上：SHOIQ 的推理是 NEXPTIME，而这篇的用途（导出元数据文件、做关键词搜索）**完全不需要推理**。这里可能是建模时随手用了强构造子，没有考虑推理代价——而既然不推理，那这个表达力标记就只是个副产品。

四、micro-ontology 这个词是名词膨胀。看图 4 那段 Turtle：里面**没有任何公理**，全是实例断言（`ns:AIModel`、`ns:hasName "..."`）。它是一份 **ABox 片段 / RDF 元数据文件**，不是本体。叫它 micro-ontology 会让读者以为拿到的是可推理的东西，实际拿到的是一份带 IRI 的键值清单。这个措辞问题在语义网领域很常见，但值得点出来。

五、概念清单缺了最难的那几项。14 个概念里，Name、Description、Author 是任何元数据方案都有的，真正有区分度的只有 Task/sub-task（而它是复用 ITO 的）、Quality criterion + Score、Training-/test-split、Data points。而恰恰是这些没有：

- 数据集的许可与合规（工业供应链交换的第一道门槛）
- 偏见与公平性指标

- 训练环境与依赖版本（这正是 EMMM 有、它没有的）
- 模型的适用边界与已知失效模式（Google 2019 年的 Model Cards 把这一项放在很前面）

对比 Model Cards 和 Hugging Face 的 model card 规范，AIMDEO 覆盖的是更窄的一块。作者自己在未来工作里提了「扩展 EMMM 以建立兼容性」，某种意义上是承认了这个缺口。

对我的启示

一、四个协作场景给了我 owner 之外的第二个维度。 昨天我给统一模型 spec 加了 owner 字段（平台内置 / 企业自定义 / AI 生成），它答的是「谁造的」。这篇的四场景答的是另一个问题——「怎么共享」：只在本企业内用、数据可共享但定义不共享、定义可共享、只通过服务暴露。

这个维度我正好需要。医药那条线上，我已经定了 K1 疾病 / K5 医疗体系 / K7 法规可跨企业共享、K2 产品 / K3 证据 / K4 竞品企业私有——这就是「共享定义」和「内部使用」两档。建议给 knowledge_unit 加一个 sharing_scope 字段，取值就照这四档。它和 owner 不重复：一个企业自定义的知识单元也可能是可共享的（比如某企业整理的疾病机制），一个平台内置的也可能因合规不可共享。

二、Individual count = 5 这个警示直接对着我。 我的 spec 现在也是纯 TBox——四张表的设计加一套字段映射，8 个 App 全都只做到「字段映射表」层面的对齐，没有任何一个 App 的数据真正建表灌进去过。这和这篇 47 个类配 5 个个体是同一个形状。

设计得再齐整，不灌数就不知道哪里不对。 具体行动：选 Roleplay 医药（数据最全、CN 有 75 家医药企业 1,805 行）真的把它的语义点、评分要点、dws_section_message_stat 灌进四张表，用它来验模型。灌的过程中会暴露的问题，靠读字段清单是想不出来的。

三、「applicability 而不是 effectiveness」这个区分要用在我自己的报告里。 我给统一模型写的每条设计判断，现在都是 applicability 级别的——「这四张表能容纳这个产品的字段」。要到 effectiveness 得回答另一个问题：用统一模型之后，跨产品查询能答出原来答不出的什么问题？

这正好接上 RTSKG 那篇给我的启示（competency questions 是本体的验收判据）。两条合起来是一条完整的路径：① 先写一组具体的 CQ（可执行的查询粒度，不是「模型要回答什么」这种元问题）→ ② 灌一个 App 的真实数据 → ③ 看这些 CQ 能不能真的答出来。答不出来的那条，就是模型还缺的东西。这比继续往 spec 里加第 9 个、第 10 个 App 更有价值。

四、半自动检测 + 人工核对，是我做语义点归类时该用的交互形态。 这篇的注册流程是：平台自动检测特征 → 用户核对并补元数据。理由是自动检测「降低出错的可能，例如避免遗漏特征」——注意它给的理由是「防漏」不是「省事」，这个定位很准：自动化保证覆盖，人保证正确。

对应到我下一步：从 CN 医药企业的 8,305 个去重语义点归类到知识域（轴 A）和知识类型（轴 C），不能全自动。应该产出一份带候选归类和置信度的清单，让人逐条核。而且自动那一步的价值主要在「不漏」——把所有语义点都过一遍、都给一个候选，而不是挑好判的做。

五、「本体的用处是规定必须填哪些字段」这句话，我可以直接拿去回答一个必然会被问到的问题。 当我把统一模型的四张表拿给别人看，一定有人问「为什么不用一张宽表加 JSON 列」。答案就是这句：宽表加 JSON

能装下任何东西，代价是接收方不知道该期待哪些 key、也不知道某个 key 的值该怎么解释。这篇拿 EMMM 做的对比就是这个论证的现成例子——EMMM 的核心概念建得不错，但元数据用任意 key-value，于是「缺乏共同格式和显式语义」。

这一条对我特别贴切，因为我昨天刚在 UMU 的代码里见过同一个病：ManaSim 的子能力分数存在 ucs MySQL 的一个 **JSON 列**里，得靠 Flink 的一个 UDTF（`AbilityScoreExplodeFn`）在数仓侧拆开、再 lookup 字典表拿 id 才能用。如果一开始就是结构化的，那个 UDTF 就不必存在。

文中金句

"The platform eases collaborative AI model development while the ontology can help to ensure that required information is available, and how this information can be provided." 平台简化了协作式 AI 模型开发，而**本体能帮助确保所需信息是可得的、以及这些信息该如何提供**。

"A predominant issue in this regard is a lack of structured metadata of AI models with explicit semantics and a common understanding of such a metadata format." 这方面的一个主要问题是：**缺少带显式语义的 AI 模型结构化元数据，也缺少对这类元数据格式的共同理解**。

"While the metamodel covers central concepts, element metadata is captured using arbitrary key-value mappings, which lacks a common format and explicit semantics." 虽然这个元模型覆盖了核心概念，**元素的元数据却是用任意的键值映射捕获的，因而缺乏共同格式和显式语义**。

"Distributed and shared use can only be achieved through a clear understanding of the functional scope of AI models." **分布式的、共享的使用只能通过对 AI 模型功能范围的清晰理解来实现**。

"Leveraging Django's Model-View-Template (MVT) architecture, any updates to the ontology in the AIMDEO can seamlessly apply to the model component of the server without disrupting its overall functionality." 借助 Django 的 MVT 架构，AIMDEO 本体的任何更新都能**无缝应用到服务端的 model 组件，而不打断其整体功能**。

全文翻译

摘要

Recently, there has been a great deal of research into improving AI methods and their application. The main focus is on tracking progress, enabling transparent comparisons, and fostering a more profound understanding of AI. In that process, different organizations generate and use plenty of assets that need to be tracked, traced and managed. Moreover, it is important to discover assets relevant for the task at hand.

近来有大量研究致力于改进 AI 方法及其应用，主要关注点是[追踪进展、支持透明比较、以及促成对 AI 更深入的理解](#)。在这个过程中，不同组织产生并使用了大量需要被追踪、溯源和管理的资产。此外，[发现与当前任务相关的资产](#)也很重要。

This paper presents research aiming to contribute to answering the question of what is required to exchange and manage AI models and related assets effectively without semantic gaps in an industrial context. We introduce a platform for AI model exchange, which facilitates the usage, exchange, and analysis of AI models and datasets. The platform incorporates an ontology that can foster a more profound common understanding of what is required in these tasks and help tackle the issues mentioned above. Finally, we elucidate the utility of the platform through the illustration of a use case in the context of real-time critical systems.

本文旨在回答这个问题：[在工业语境下有效交换和管理 AI 模型及相关资产、且不产生语义鸿沟，需要什么？](#)我们提出一个 AI 模型交换平台，便利模型与数据集的使用、交换和分析。该平台内置一个本体，可以促成对「这些任务需要什么」的更深入的共同理解，并帮助应对上述问题。最后我们通过一个实时关键系统语境下的用例来阐明平台的效用。

关键词：人工智能、语义网、链接数据、本体、语义标注。

1. 引言：模型转手之后，接收方看不懂它

The widespread application and sheer number of existing Artificial Intelligence (AI) models demand discovering the right assets and building trust in the end-user using the models. By the time of this writing, the data collection and development of AI models is often separated from the final users of the models. However, distributed and shared use can only be achieved through a clear understanding of the functional scope of AI models. This requires a characterization of AI models, which includes basic descriptions of the models and basic information about the data, quality, and context.

AI 模型的广泛应用与庞大数量，要求能[发现正确的资产](#)并在[使用模型的终端用户那里建立信任](#)。在本文写作之时，数据采集与 AI 模型开发往往与模型的最终使用者相分离。然而，[分布式的、共享的使用只能通过对 AI 模型功能范围的清晰理解来实现](#)。这需要对 AI 模型做刻画，包括模型的基本描述，以及关于数据、质量和上下文的基本信息。

In the automotive industry, for instance, AI models and related assets are transferred between different tiers along the supply chain. A predominant issue in this regard is a lack of structured metadata of AI models with explicit semantics and a common understanding of such a metadata

format. Popular AI communities, like Hugging Face, for example, host numerous AI models and datasets. Information about these resources is provided by semi-structured descriptions, mainly consisting of natural language text of differing kinds of content and amounts. While some but not all models have structured metadata, the lack of them hinders the shared use and development of AI models.

举例来说，在汽车行业，AI 模型和相关资产沿供应链在不同层级（tier）的供应商之间转移。**这方面的一个主要问题是：缺少带显式语义的 AI 模型结构化元数据，也缺少对这类元数据格式的共同理解。**像 Hugging Face 这样流行的 AI 社区托管了大量 AI 模型和数据集，而关于这些资源的信息是由**半结构化描述**提供的——主要由内容种类和篇幅各异的自然语言文本构成。**有些模型有结构化元数据，但并非全部；元数据的缺失阻碍了 AI 模型的共享使用和开发。**

Current state-of-the-art machine learning platforms such as MLflow, H2O and Ray have means to add metadata to models. However, the metadata specifications lack interoperability. This problem becomes even more apparent when models need to be exchanged among different companies due to differing metadata content or formats.

当前最先进的机器学习平台如 MLflow、H2O、Ray 都有给模型添加元数据的手段。**然而这些元数据规范之间缺乏互操作性。**当模型需要在不同公司之间交换时，由于元数据内容或格式各异，这个问题变得更加明显。

主要贡献三条：一个协作式 AI 开发与使用的平台；一个描述 AI 模型与数据集的**本体**；一个展示该方法**可用性**（applicability）的用例。

2. 相关工作

2.A AI 资产管理。 The authors in [1] assessed various tools and platforms for machine learning asset management. Of all the options, only MLFlow supports asset registry and exchange without requiring data sharing with third parties. However, it does not support ontologies for the description of metadata of assets. MLEM is an open-source tool that provides a standard interface for deploying machine learning models and offers a model registry. However, its model registry is very limited and does not collect metadata. Hugging Face is a hub for machine learning models and datasets and offers an interface for deploying the models.

文献 [1] 评估了多种机器学习资产管理工具和平台。**在所有选项中，只有 MLflow 支持资产注册与交换而不要求把数据共享给第三方；然而它不支持用本体描述资产的元数据。**MLEM 是一个开源工具，为部署机器学习模型提供标准接口并提供模型注册表；**然而它的模型注册表非常有限，且不收集元数据。**Hugging Face 是机器学习模型与数据集的中心，并提供模型部署接口。

2.B AI 资产刻画。 Recent work from Blagec et al. introduced the Intelligence Task Ontology and Knowledge Graph (ITO) which aims primarily to study scientific research but can also be used to annotate and organize information in the AI domain. While the ITO has the purpose of describing AI tasks, the AIMDEO ontology presented in this paper is meant to support the exchange of AI

models and datasets. Moreover, in contrast to the ITO, the AIMDEO contains concepts for describing the provenance and kind of AI model as well as parameters of AI models and datasets. Additionally, the ontology presented here allows specifying model evaluation metrics. Both, the ITO and the AIMDEO support the specification of intended tasks and subtasks of AI models and datasets.

Blagec 等近期的工作提出了**智能任务本体与知识图谱 (ITO)**，其主要目标是研究科学研究本身，但也可用于标注和组织 AI 领域的信息。ITO 的目的是描述 AI 任务，而本文提出的 AIMDEO 意在**支持 AI 模型与数据集的交换**。此外，与 ITO 不同，AIMDEO 包含描述 **provenance (来源)** 和 **AI 模型种类**、以及**模型与数据集参数**的概念；本文的本体还允许**指定模型评估指标**。两者都支持规定 AI 模型与数据集的预期任务和子任务。

Idowu et al. presented the Experiment Management Meta-Model (EMMM), which is a metamodel of asset types and their relationships common to the management of machine learning experiments. Besides modeling central concepts such as models, parameters, or dependencies, this metamodel enables the specification of arbitrary metadata for any asset. In addition to the asset structures and their relationships, the metamodel considers version control structures for machine learning as well as traditional assets. While the metamodel covers central concepts, element metadata is captured using arbitrary key-value mappings, which lacks a common format and explicit semantics.

Idowu 等提出了**实验管理元模型 (EMMM)**，它是机器学习实验管理中常见的资产类型及其关系的元模型。除了建模 models、parameters、dependencies 这类核心概念，该元模型还允许为任何资产**指定任意元数据**。除资产结构及其关系之外，该元模型也考虑了机器学习资产与传统资产的版本控制结构。**虽然这个元模型覆盖了核心概念，元素的元数据却是用任意的键值映射捕获的，因而缺乏共同格式和显式语义。**

3. 交换平台

With the advancement of AI in the recent decade, a vast number of AI assets have been produced within various companies and organizations. However, these assets generally lack any additional metadata and are either not discoverable or inaccessible by others. There is also a discrepancy between the environment in which an asset is created and the environment in which other users will use it, which can lead to various compatibility issues and hinder the usage of assets.

随着近十年 AI 的推进，各公司和组织内部产生了海量 AI 资产。**然而这些资产通常缺少任何额外元数据，要么无法被发现、要么无法被他人访问。**此外，**资产被创建的环境与其他用户将要使用它的环境之间存在差异**，这会导致各种兼容性问题并阻碍资产的使用。

To address the abovementioned issues, we have developed an AI Model and Dataset Exchange Platform (AIMDEP), which provides a central registry to make AI models and datasets visible and accessible to all users. The AIMDEP uses the AIMDEO to specify the assets and captures additional metadata for each asset accordingly to make them more interpretable. Moreover,

AIMDEP supports the export of AI model metadata according to the AIMDEO in the form of micro-ontologies. Finally, the online deployment integrated into the AIMDEP provides a standard platform-independent interface to deploy the assets.

为解决上述问题，我们开发了 **AI 模型与数据集交换平台（AIMDEP）**，它提供一个中央注册表，使 AI 模型和数据集对所有用户可见可访问。AIMDEP 用 AIMDEO 来规定资产、并相应地为每个资产捕获额外元数据以使其更可解释。此外，AIMDEP 支持按 AIMDEO 以「微本体」形式导出 AI 模型元数据。最后，集成在 AIMDEP 中的在线部署提供了一个平台无关的标准接口来部署资产。

The platform employs a client-server architecture, with the server and the Web User Interface (UI) implemented using the Django framework. Additionally, custom clients can interact with the server through the provided REST API. Leveraging Django's Model-View-Template (MVT) architecture, any updates to the ontology in the AIMDEO can seamlessly apply to the model component of the server without disrupting its overall functionality. This design ensures the platform's flexibility for accommodating future ontology updates. The communication between the server and the clients is encrypted, and the access management policy is implemented on the server side to restrict access to assets to privileged users.

平台采用客户端-服务器架构，服务端与 Web 用户界面用 Django 框架实现；自定义客户端可通过所提供的 REST API 与服务端交互。**借助 Django 的 MVT（Model-View-Template）架构，AIMDEO 本体的任何更新都能无缝应用到服务端的 model 组件，而不打断其整体功能。这一设计确保了平台容纳未来本体更新的灵活性。**服务端与客户端之间的通信是加密的，访问管理策略在服务端实现，把资产访问限制在有权限的用户。

3.A 资产注册。 The registration of models and datasets starts by uploading the physical asset files to the platform. The AIMDEP attempts to identify the back-end required to access and deploy the asset. This includes identifying the data handlers for the datasets and the framework needed to evaluate and deploy the model. The AIMDEP supports various common dataset formats, including CSV, Excel, JSON, and Parquet, and offers support for models exported by the most popular machine learning frameworks, including Scikit-Learn, TensorFlow, and PyTorch. The user can edit the selected framework if required. Any custom dataset and model can also be registered without specifying the framework. However, the platform would be unable to visualize the dataset or deploy the model later, since the platform requires the framework to interact with the assets.

模型和数据集的注册从把物理资产文件上传到平台开始。**AIMDEP 尝试识别访问和部署该资产所需的后端——包括为数据集识别数据处理器、为模型识别评估和部署所需的框架。**AIMDEP 支持 CSV、Excel、JSON、Parquet 等常见数据格式，并支持由 Scikit-Learn、TensorFlow、PyTorch 等最流行框架导出的模型。**用户可以修改平台选中的框架。任何自定义数据集和模型也可以在不指定框架的情况下注册，但平台之后将无法可视化该数据集或部署该模型——因为平台需要框架才能与资产交互。**

Next, models and datasets' input and output features are defined semi-automatically. The platform tries to detect the features, but the user should verify and add further metadata for each feature. Each feature is then stored as a Parameter based on AIMDEO. The user should also add the configuration parameters of models at this stage. Finally, the user adds the additional metadata following the AIMDEO, which includes, for instance, adding metrics to the model.

接下来，模型和数据集的输入输出特征以**半自动**方式定义：**平台尝试检测特征，但用户应当核对并为每个特征补充元数据**。每个特征随后按 AIMDEO 存为一个 `Parameter`。用户也应在此阶段添加模型的配置参数。最后，用户按 AIMDEO 补充其余元数据，例如给模型添加评估指标。资产随后即可注册进平台的中央数据库。

3.B 资产操作。 For datasets, it offers interactive visualization in the form of statistical tables, various plots, and feature analyses (only for numerical datasets) to identify the importance of each feature. Since the visualization of the whole dataset can be very time-consuming for massive datasets, the AIMDEP can limit the visualization to a random subset of the dataset with a given size.

对数据集，平台提供统计表、多种图表、以及特征分析（仅对数值型数据集）形式的交互式可视化，用来识别各特征的重要性。**由于对海量数据集做全量可视化非常耗时，AIMDEP 可以把可视化限制在给定规模的随机子集上。**

Online deployment is offered for the registered models with a supported framework. It enables a simple and platform-independent interface for the inference of the models. The inference of a model can be performed either through the REST API or through a graphical interface based on Gradio, which is generated based on the task and subtask of the model and its input and output features.

对使用受支持框架的已注册模型提供在线部署，为模型推理提供简单且平台无关的接口。模型推理可以通过 REST API 完成，也可以通过**基于 Gradio 的图形界面**——该界面是**根据模型的 task 与 subtask、以及它的输入输出特征自动生成的**。

The search is powered by OpenSearch and utilizes the metadata captured for each asset to perform a search and returns the most relevant assets for the query. Assets can also be downloaded for offline use or integration in custom workflows. Next to the original files, additional metadata captured for the asset is also provided as micro-ontologies containing the attributes and their values.

搜索由 OpenSearch 驱动，利用为每个资产捕获的元数据执行搜索，返回与查询最相关的资产。资产也可以下载用于离线使用或接入自定义工作流。**除原始文件之外，为资产捕获的额外元数据也以「微本体」形式提供，其中包含属性及其取值。**

4. 本体

The AI Model and Dataset Exchange Ontology (AIMDEO) captures essential concepts that are part of collaborative AI model development and use. The ontology is expressed as an OWL ontology.

AI 模型与数据集交换本体 (AIMDEO) 捕获协作式 AI 模型开发与使用中的核心概念，用 OWL 表达。（本体指标见「核心内容」的表。）

4.A 协作场景。 We identified different collaboration scenarios to get a more profound understanding of concepts required to capture all information necessary in collaborative AI model development and use.

我们识别了不同的协作场景，以更深入地理解「要捕获协作式 AI 模型开发与使用中所有必要信息，需要哪些概念」。

1) 内部使用：AI 模型和数据集留在创建者手里。模型在 AI 开发环境内被使用（例如在汽车行业的某一层供应商内部），结果随后存进产品开发环境，反馈再返回 AI 开发环境。

2) 共享数据：如果数据无法内部采集，在供应链参与方之间交换数据是一个选项。此场景下，开发数据库中的数据可被用于 AI 开发环境。**这种协作形式要求对数据访问有描述和良定义的接口。**

3) 共享模型：另一种合作形式是共享训练好的模型。与前一种形式相反，**产品开发环境的数据不被共享，而是共享 AI 模型。**

4) 模型即服务：第四种也是最后一种合作场景基于服务的部署与使用。**不直接交换 AI 模型，用户把相应请求发给一个服务，该服务在内部委托给 AI 模型。**

4.B 本体概念。 Creating the ontology requires identification of relevant concepts and their relationships. These are then modeled in the ontology as corresponding OWL classes and properties.

创建本体需要识别相关概念及其关系，然后把它们在本体中建模为相应的 OWL 类和属性。（14 个核心概念的三组分类见「核心内容」。）

5. 评估：一个用例走查

This section evaluates the proposed approach with the help of a collaboration scenario where typical stakeholders, like domain experts, AI experts, and end users, work interdependently and interdisciplinary together. The use case covers the development and usage of an AI model that facilitates the prediction of memory access time, which is essential in predicting software timing in safety-critical applications.

本节借助一个协作场景评估所提方法——领域专家、AI 专家、终端用户这些典型利益相关方相互依赖、跨学科地协同工作。用例涵盖一个**预测内存访问时间**的 AI 模型的开发与使用，这在**安全关键应用的软件时序预测**中是必需的。

The dataset is created by a domain expert or, in our case, a hardware developer. The domain expert describes his inherent knowledge using our proposed ontology AIMDEO. In our use case, this covers cache (memory hierarchy) characteristics, such as replacement strategy and size. The dataset is registered on the AIMDEP, easing the Dataset Exchange. The AI expert can access the dataset and the knowledge specified by the domain expert. This supports the AI expert in designing a suitable AI model.

数据集由领域专家（本例中是硬件开发者）创建。**领域专家用我们提出的 AIMDEO 本体把他的隐性知识描述出来**——本例中涵盖缓存（内存层次）特性，如替换策略和大小。数据集注册到 AIMDEP 上，便利了「数据集交换」。AI 专家能够访问数据集以及领域专家所规定的那些知识，这支持 AI 专家设计合适的 AI 模型。

An end user, in our case a software developer, is looking for an AI model to predict software timing. The developer looks for a suitable memory configuration for his or her time-sensitive software for embedded systems. He or she can use the keyword search provided by the platform to find an appropriate model. The found model can then be used to analyze his software with the memory configuration, as described using the ontology, thus assessing the purchase of test hardware.

终端用户（本例中是软件开发者）在寻找一个预测软件时序的 AI 模型，为他的嵌入式系统时间敏感软件寻找合适的内存配置。他可以用平台提供的关键词搜索找到合适的模型，然后用该模型结合本体所描述的内存配置分析自己的软件，**从而评估是否要采购测试硬件**。

5.A 数据集交换。 The platform recognizes all features semi-automatically when the dataset is uploaded, making it easier to describe them. One advantage of automatically recognizing all features of supported data formats is reducing susceptibility to errors, e.g., by forgetting features. In combination with the platform's input mask, the user is also actively supported when entering information, such as a description and the value range.

数据集上传时平台半自动识别所有特征，使描述它们更容易。**自动识别受支持数据格式的所有特征的一个好处是降低出错的可能，例如避免遗漏特征。**结合平台的输入模板，用户在录入描述、取值范围这类信息时也得得到主动支持。

The user does not have to download the dataset manually to analyze it; instead, they can use the analysis functions provided by the platform. This allows for the rapid analysis, comparison, and selection of a suitable dataset, which can then be downloaded together with the description of the dataset by the ontology. This process can also be accomplished via a REST API, enabling easy integration with other tools.

用户不必手动下载数据集就能分析它，可以直接用平台提供的分析功能。这支持快速的分析、比较和选型，选定后可以连同本体对该数据集的描述一起下载。这个过程也能通过 REST API 完成，便于与其他工具集成。

5.B AI 模型交换。 Once the AI expert has developed a suitable AI model, it can be published on the platform. Additional information is added to the model using the ontology to enable the end

user to quickly and easily assess the model's suitability for the specific application. Firstly, the input and output parameters are described and named. For this purpose, the name, a description, the data type, and optionally, a valid value range and the distribution are specified for each feature.

AI 专家开发出合适的模型后即可发布到平台上。用本体给模型补充额外信息，使终端用户能快速便捷地评估该模型对特定应用的适合度。首先描述并命名输入输出参数——为每个特征指定名称、描述、数据类型，以及可选的**有效取值范围和分布**。

With the evaluation information, end users can also assess the quality of the model more efficiently and better, such as using the quality metric and the breakdown of the dataset for training and testing. Another essential piece of information is the dataset specification used to assess better the model's suitability for the end users' data. By linking the dataset, the user can also find out what format and quality the training data for the model provided was in and whether their data matches it.

有了评估信息，终端用户能更高效更好地评估模型质量，例如借助质量指标以及数据集的训练/测试划分。**另一项关键信息是数据集规格**，用来更好地评估模型对终端用户自己数据的适合度。**通过链接数据集，用户还能了解该模型的训练数据是什么格式、什么质量，以及自己的数据是否与之匹配。**

The platform provides execution of the model for different frameworks, which is directly done via the platform. The fast execution makes trying out the model for end users' data possible without setting up a suitable runtime environment. This lowers the hurdle for using and experimenting with an AI model.

平台提供不同框架下的模型执行，直接在平台上完成。**快速执行使终端用户能在自己的数据上试用模型，而无需搭建合适的运行时环境。这降低了使用和试验一个 AI 模型的门槛。**

6. 结论

We used the AIMDEP successfully to collaboratively develop AI models. However, its application is not limited to that. It is possible to use either the platform or the ontology separately. The platform eases collaborative AI model development while the ontology can help to ensure that required information is available, and how this information can be provided. The metadata specification and export functionality of the platform can help in integrating different tools required in the AI model development process.

我们成功地用 AIMDEP 协作开发了 AI 模型，但它的应用不限于此。**平台和本体可以分开使用：平台简化了协作式 AI 模型开发，而本体能帮助确保所需信息是可得的、以及这些信息该如何提供。** 平台的元数据规定与导出功能可以帮助整合 AI 模型开发过程中所需的不同工具。

Future work could, for instance, investigate on extending EMMM to create a certain degree of compatibility. The generic key-value pair metadata associated with the resource types in the

metamodel could be used to store AIMDEO metadata. Creating further interoperability between AIMDEO and ITO might also be worth investigating.

未来工作可以研究[扩展 EMMM 以建立一定程度的兼容性](#)——元模型中与资源类型关联的通用键值对元数据可以用来存储 AIMDEO 元数据。在 AIMDEO 与 ITO 之间建立进一步的互操作性也可能值得研究。

本文由德国联邦经济事务与气候行动部（BMWi）在 progressivKI 项目下资助（资助号 19A21006M）。

参考链接

- [DOI 10.1109/INDIN58382.2024.10774524](#) — 正式发表版（IEEE INDIN 2024, pp. 1 - 6）。arXiv 上这份是 accepted manuscript
- [Hugging Face](#) — 论文的主要对照物：托管大量模型与数据集，但资源信息是半结构化的自然语言描述
- [MLflow](#) — 唯一支持「资产注册与交换且不需要把数据交给第三方」的既有平台，但不支持用本体描述元数据
- [MLEM](#) — 平台用它做在线部署；论文同时指出它自带的模型注册表很有限、不收元数据
- [ITO 标识符示例 ito:ITO_42033](#) — AIMDEO 直接复用 ITO（智能任务本体）的 IRI 来标注模型的 task 与 subtask，是「复用现有词汇」的实例
- [Gradio](#) — 平台据模型的 task/subtask 与输入输出特征自动生成推理界面
- [OpenSearch](#) — 驱动平台的语义搜索，检索的是各资产被捕获的元数据
- [Plotly](#) — 服务端渲染数据集的统计表与图表